phepex

Fast DSP and pulse extraction for PMT/SiPM waveforms

C++ kernels · Python bindings · Extracted from CTAO libdvr

F. Werner (MPIK) · github.com/phepex/phepex

Why phepex?

A small package for a very common waveform-processing problem

- CTA-style events contain blocks of many waveforms.
- For many workflows, the expensive part is DSP and pulse extraction.
- `libdvr` already contained optimized extraction code.
- `phepex` factors that functionality out into a focused library.
- The result: easier reuse from both C++ and Python.

For ctapipe users: phepex focuses on low-level waveform and pulse extraction, rather than full event processing.

Relationship to libdvr

phepex is the reusable DSP and pulse-extraction core

libdvr

- CTAO data volume reduction library.
- Contains event-level DVR logic.
- Originally included DSP and pulse-extraction functionality.
- Optimized for production-style reduction workflows.

phepex

- Splits out the DSP and pulse-extraction pieces.
- Keeps them independent and easier to test.
- Adds Python bindings.
- v0.2 brings significant speed-ups.

phepex is now the extraction engine of libdvr.

Relationship to ctapipe

Designed to fit the mental model ctapipe users already have

Waveforms in

channel × pixel × sample arrays (uint16 or float)

Signal processing

upsampling, deconvolution, filtering

Features out

neighbor summation, pulse time & integral

Downstream use

cleaning, reconstruction, monitoring, ...

ctapipe provides

- Event sources.
- Calibration machinery.
- Containers and pipeline structure.
- Higher-level analysis components.

phepex focuses on

- Fast low-level waveform operations.
- Pulse extraction algorithms.
- C++ performance with Python access.
- Reusable kernels that can be embedded elsewhere.

What is inside?

A focused set of DSP and extraction building blocks

- C++ implementation of waveform-processing kernels.
  - `preprocess_waveform(s)` – upsampling, deconvolution, smoothing.
- Pulse extraction routines derived from the libdvr work.
  - `neighbor_peak_indices`, `adaptive_centroid`.
- Python bindings for interactive, analysis, and pipeline use.
  - Plus a `FastFlashCamExtractor` drop-in for ctapipe.
- Minimal dependency surface; straightforward to embed.
- Compact library boundary: no full event model required.
- Useful for benchmarking extraction choices outside a full pipeline.

Package goal

Make the reusable part of waveform processing fast, and easy to call, test and compare.

v0.2: speed-ups

Beyond factoring out and wrapping the libdvr kernels

- Faster waveform processing and pulse extraction for waveform blocks.
  - SIMD-friendly tiled processing + CPU pipeline saturation.
- Zero-copy Python interfaces for both uint16 (R1/DL0) and float32 (ctapipe) data.
- Lower barrier to benchmarking extraction variants.

FlashCam upsampling/deconvolution

scipy/numpy	980 µs/op
phepex (Python)	134 µs/op
fc-utils (C)	147 µs/op
phepex (C++)	96 µs/op

+630% (Python) / +53% (C++)

`phepex/benchmarks/cpp/microbench.cpp`

ctapipe throughput

ctapipe `FlashCamExtractor`	370 ev/s
`FastFlashCamExtractor`	1235 ev/s

+233%

`phepex/benchmarks/benchmark-fast-extractor.py`

libdvr throughput (incl. smoothing)

before refactor	1338 ev/s/thread
with phepex 0.2	2175 ev/s/thread

+62%

`libdvr/cmd/dvr-benchmark/main.cpp`

Measured on an Apple M1 Pro for a typical FlashCam event size (1764 pixels, 22 samples, uint16).

Outlook

Testing/integration + plans

- Verify `FastFlashCamExtractor` with end-to-end time/charge resolution curves.
- Integrate ‘new’ FlashCam saturation recovery & time extraction algorithms.
- Create PR into ctapipe.
  - All cameras will benefit from the drop-in routines.
- Tuning for DL0 (zero-suppressed) data model.
- Support for sample-level per-channel time offsets (LSTCam).

Takeaway

phepex makes the PMT/SiPM signal extraction core reusable

- Extracted from libdvr’s DSP and pulse-extraction functionality.
- Packaged as a focused C++ library.
- Exposed to Python for ctapipe-style analysis workflows.
- v0.2 adds important speed-ups.
- Useful wherever waveform extraction should be fast, testable, and independent.

Resources

- Repository: github.com.phepex/phepex
- Documentation: phepex.github.io/phepex